

AI-BASED RESUME SCREENING SYSTEM

INSTRUCTIONS:

1. Read carefully and understand the Project.
2. After completion of the project upload it to GitHub and fill up your record on below link.
3. If any student from the “non-technical” background you can take the help of the other students for web-based framework like flask or Django or you can use html, CSS and java for completion of the project.

You can directly use this for:

- Resume project section
- Interview explanation
- GitHub repository
- Mini capstone project

1) FULL PROJECT ARCHITECTURE

System Architecture Flow

1. Resume Upload (PDF/DOC)
2. Job Description Input
3. Resume Parsing Module
4. Text Preprocessing Module
5. Feature Engineering / Embeddings
6. Similarity Computation
7. Ranking Engine
8. Results Visualization (Web UI)
9. Optional LLM Explanation Module

Component-Level Architecture

A. Input Layer

- Resume Upload (multiple files)
- Job Description input box

B. Document Parsing Layer

Libraries:

- pdfplumber / PyMuPDF
- python-docx
- Apache Tika

Purpose:

- Convert document into plain text

C. NLP Preprocessing

Steps:

1. Lowercasing
2. Removal of special characters
3. Stop word removal
4. Lemmatization
5. Skill normalization

Libraries:

- spaCy
- NLTK
- re

D. Feature Representation

Choices:

- TF-IDF (Baseline)
- Sentence Transformers / BERT (Advanced)

Use:

- Resume embeddings
- Job description embedding

E. Similarity Engine

Metric:

- Cosine similarity

Logic:

Score(resume, JD) = cosine(Resume_embedding, JD_embedding)

F. Ranking Engine

- Sort by similarity score
- Apply threshold filter

G. Output Layer

1. Ranked table
2. Match percentage
3. Top skills found
4. Missing skills

H. Front-End (Optional but Recommended)

Framework:

- Streamlit / Flask

2) MODEL SELECTION GUIDANCE

A. Embedding Model Choices

Beginner / Fast

- TF-IDF + Logistic Regression
- Cosine similarity

Intermediate

- Word2Vec / GloVe

Advanced (Recommended)

Sentence Transformers:

- all-MiniLM-L6-v2
- paraphrase-MiniLM-L3-v2

- distilbert-base-nli-stsb

Why:

- Context aware
- Excellent for semantic matching

B. Optional Classifier Model

If you want classification ("Shortlist" vs "Reject"):

- Random Forest
- XGBoost
- Logistic Regression

Input:

- Similarity score
- Skill count
- Experience length

C. Explainability Models (OPTIONAL)

- SHAP
- LIME

Why:

Explain why a resume was ranked high/low.

3) PYTHON PROJECT FOLDER STRUCTURE

AI_Resume_Screening/

```
|
|
|  └─ data/
|
|    └─ resumes/
|    └─ job_descriptions/
|
|  └─ models/
|    └─ embedding_model/
|
|  └─ src/
|    └─ parser.py
|    └─ preprocess.py
|    └─ vectorizer.py
|    └─ matcher.py
|    └─ ranker.py
|    └─ skill_extractor.py
|
```

└─ app.py # Streamlit / Flask

|

└─ utils/

| └─ helpers.py

|

└─ requirements.txt

└─ README.md

4) DATASET RECOMMENDATIONS

Resume Datasets (Students can use any one of the below dataset)

Search on Kaggle:

- "Resume Dataset"
- "Recruitment Dataset"
- "CV/Resume Text Dataset"
- "Job Description Dataset"

Examples:

- Resume Entities Dataset
- Resume Ner Dataset
- Job Description Classification Dataset

Job Description Sources

- Indeed
- LinkedIn Jobs
- Free API job boards

Skills Dataset

- O*NET Skills Database
 - ESCO Skill Taxonomy (EU)
 - GitHub skill lists
-

5) INTERVIEW TALKING POINTS (VERY IMPORTANT) (Not the part of project process)

Use these answers directly in interviews.

Q1: Why did you choose this project?

"I wanted to solve a real-world hiring problem involving large-scale text processing and NLP. This project uses AI to automate resume screening, which is extremely relevant in HR tech."

Q2: Why BERT/Sentence Transformers?

"TF-IDF fails to capture semantic similarity. Sentence Transformers understand contextual meaning, which makes resume-job matching more realistic."

Q3: How do you handle mismatch in skills?

"I normalize skill terms using a skill dictionary and extract top skills using SpaCy. I also perform fuzzy matching."

Q4: How ranking is done?

"I compute cosine similarity between resume embedding and job embedding, and sort candidates by descending score."

Q5: Bias reduction?

"I avoid personal identifiers and base decisions only on professional content like skills and experience."

Q6: Performance optimization?

"I cache embeddings and batch encode resumes for faster processing."

6) GITHUB README TEMPLATE

You can copy and paste this directly into your repo.

AI-Based Resume Screening System

Description

An intelligent resume screening system using NLP and Machine Learning to automatically rank job candidates based on skill match, relevance, and job similarity.

Features

- | | |
|--|---|
| <ul style="list-style-type: none">• Resume Parsing• Skill Extraction• Semantic Matching• Ranking Engine | <ul style="list-style-type: none">• Candidate Shortlisting• Web Interface (Streamlit)• Explainable AI |
|--|---|

Tech Stack

- | | |
|--|---|
| <ul style="list-style-type: none">• Python• NLP (spaCy, NLTK)• ML (scikit-learn) | <ul style="list-style-type: none">• Transformers• Streamlit• Pandas |
|--|---|

Workflow

- | | |
|-------------------------|------------------------------|
| 1. Resume Upload | 4. Similarity Scoring |
| 2. Preprocessing | 5. Ranking and Visualization |
| 3. Embedding Generation | |

Installation

pip install -r requirements.txt

Usage

streamlit run app.py

Models Used

- | | | |
|---|--|---|
| <ul style="list-style-type: none">• Sentence Transformers | <ul style="list-style-type: none">• Cosine Similarity• TF-IDF Baseline (optional) | Example Output <ul style="list-style-type: none">• Rank• Candidate Name |
|---|--|---|

- | | | |
|---|---|---|
| <ul style="list-style-type: none">• Match Score• Extracted Skills• Missing Skills | Improvements <ul style="list-style-type: none">• LLM explanations• Bias detection | <ul style="list-style-type: none">• Feedback loop• Online deployment |
|---|---|---|

7) HOW TO PUT THIS ON YOUR RESUME

Project Entry:

AI-Based Resume Screening System

- Built NLP-based system to parse, embed, and rank resumes using Sentence Transformers
- Implemented cosine similarity ranking and skill extraction
- Deployed using Streamlit
- Integrated explainability module for HR transparency

PROJECT SUBMISSION LINK:



<https://docs.google.com/spreadsheets/d/1kAeb1h4e-xqS0jR4teNzRJBvOAQa3fVUZ8WmzX5cFy4/edit?usp=sharing>